Deploying Cross-Platform DLang Games on Steam

From Dev to Production

Eric Yoon

Student @ Yale

Graduating Spring 2027 with B.S. and M.S. in Computer Science

Prev. internships

PLAID Financial technology

FIZZ Consumer social

Eric Yoon

- Hobbyist game dev
- **salad** in DLang Discord!
- Indoctrinated into D in 2025

So you want to build a game in D...

^

PC

→ You're probably going to want to publish on Steam.

So you want to build a game in D...

What libraries might we want?

GUI?

Mac + PC
cross-compatibility!

DISTRIBUTION?

DRM, achievements,
game overlay

GROWTH?

Social SDK, rich presence,
network effects

…and how are we going to ship these?

Intel vs. ARM?

Windows Defender / Mac Gatekeeper?

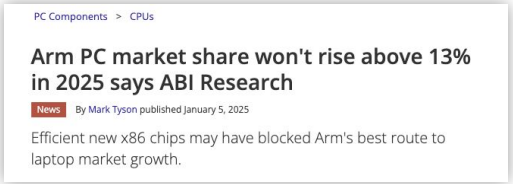
Bindings for D?

Static vs. dynamic linking?

Let’s work backwards

.APP .EXE

Windows


PC Components > CPUs

Arm PC market share won't rise above 13% in 2025 says ABI Research

News By Mark Tyson published January 5, 2025

Efficient new x86 chips may have blocked Arm's best route to laptop market growth.

Intel vs. ARM?

Windows Defender / Mac Gatekeeper?

Bindings for D?

already exist…?

DIY?

Static vs dynamic linking?

proprietary libs :/

macOS

Intel vs ARM?

:|

Windows Defender / Mac Gatekeeper?

→ $100/year, and a whole lotta xcode commands >_<

Bindings for D?

same as windows :)

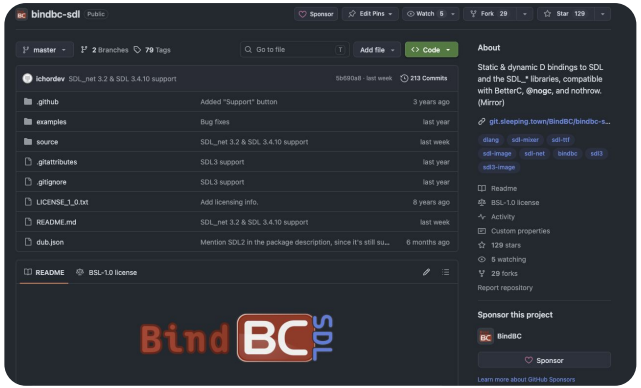
Static vs dynamic linking?

C Bindings for D

SDL3 Bindings

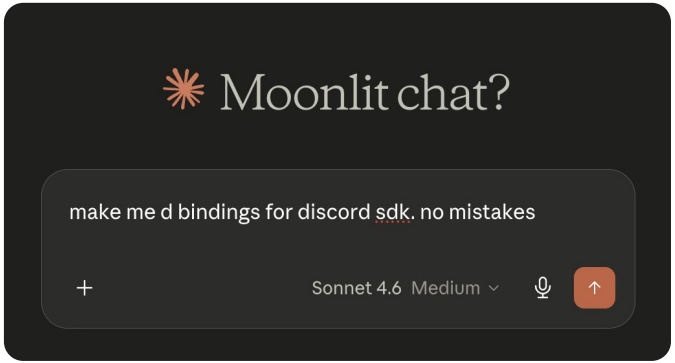
- BindBC to the rescue!
- Community-maintained BindBC extension for SDL3

  (thank you OSS maintainers!)
- Just make sure to pin SDL3 to the current bindbc-supported version



Discord & Steamworks

- …we’re on our own.
- We can use static analysis–driven bindings generators like SWIG



Discord & Steamworks

(but in all seriousness: took < 1h with LLMs!)

- Free from hallucinations (deterministic codegen)

Our desired output:

- **`discord.d`**: SWIG-generated D bindings
- **`discord_im.d`**: Finds and loads dynamic library at runtime (`dlopen`)
- **`libdiscord_wrap.dylib`**: SWIG-generated C “glue”
- **`libdiscord.dylib`**: dynamic library distributed by Discord

Define SWIG interface

```
%module discord
%{
/* Strip DISCORD_API for the C wrapper */
#define DISCORD_API
#include "cdiscord.h"
%}

/* Tell SWIG to treat DISCORD_API as empty */
#define DISCORD_API

/* Map standard C types that SWIG needs to know about */
%include "stdint.i"

/* Process the entire C header */
%include "cdiscord.h"

```

Run SWIG

```
cd /path/to/discord_social_sdk
cd include/
mkdir -p dlang_out/discord

swig -d -d2 \
    -I. \
    -outdir dlang_out \
    -o dlang_out/discord_wrap.c \
    -package discord \
    discord.i
```

Generate wrapper dylib

```
cc -arch x86_64 -shared \
    -o "$DISCORD_SDK/libdiscord_wrap-x64.dylib" \
    "$DISCORD_SDK/include/dlang_out/discord_wrap.c" \
    -I"$DISCORD_SDK/include" \
    -L"$DISCORD_SDK/lib/release" -ldiscord_partner_sdk \
    -install_name @rpath/libdiscord_wrap.dylib

cc -arch arm64 -shared \
    -o "$DISCORD_SDK/libdiscord_wrap-arm64.dylib" \
    "$DISCORD_SDK/include/dlang_out/discord_wrap.c" \
    -I"$DISCORD_SDK/include" \
    -L"$DISCORD_SDK/lib/release" -ldiscord_partner_sdk \
    -install_name @rpath/libdiscord_wrap.dylib

lipo -create \
    "$DISCORD_SDK/libdiscord_wrap-x64.dylib" \
    "$DISCORD_SDK/libdiscord_wrap-arm64.dylib" \
    -output "$DISCORD_SDK/libdiscord_wrap.dylib"
```

macOS

```
cl /nologo /LD /O2 /MT ^
    /I include ^
    /I include\dlang_out ^
    /Foobj\ ^
    /Feinclude\dlang_out\discord_wrap.dll ^
    include\dlang_out\discord_wrap.c ^
    /link lib\release\discord_partner_sdk.lib
```

windows

```
{
    "authors": [
        "Yoonicode"
    ],
    "copyright": "Public Domain",
    "description": "A minimal example of using the Discord SDK in DLang",
    "license": "Unlicense",
    "name": "discord-sdk-example",
    "lflags": [
        "-rpath", "@executable_path/lib"
    ]
}
```

dub.json

app.d

```
Discord_Client client;

void main()
{
    client = new Discord_Client();
    Discord_Client_Init(client);
    Discord_Client_SetApplicationId(client, DISCORD_APP_ID);
    Discord_Client_Connect(client);
    writeln("Discord SDK connected!");

    while(true) {
        Discord_RunCallbacks();
        Thread.sleep(1.seconds);
    }
}
```

Discord & Steamworks

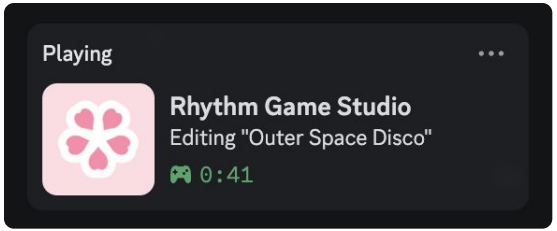
- Generated bindings available on GH

```
public void updatePresence(DiscordPresenceState state = null) {
    if (client is null) return;

    try {
        auto activity = new Discord_Activity();
        Discord_Activity_Init(activity);

        if(state !is null) {
            if(!state.details.empty) Discord_Activity_SetDetails(activity, toDiscordString(state.details));
            if(!state.state.empty) Discord_Activity_SetState(activity, toDiscordString(state.state));
        }
        auto cb = new SWIGTYPE_p_f_p_struct_Discord_ClientResult_p_void__void(cast(void*)&discordDummyCb, false);
        auto freeCb = cast(discord.discord_im.SwigExternC!(void function(void*))) &discordDummyFreeCb;

        Discord_Client_UpdateRichPresence(client, activity, cb, freeCb, null);
    } catch (Exception e) {
        writeln("Failed to update Discord presence: ", e.msg);
    }
}
```



Platform-Agnostic Code

dub.json

```
    "buildTypes": {
        "development-mac": {
            "versions": ["Development", "Platform_MacOS"],
            "lflags": [
                "-rpath", "@executable_path/../lib-snapshots/dylib"
            ]
        },
        "development-win": {
            "versions": ["Development", "Platform_Windows"]
        },
        "release-mac": {
            "versions": ["Release", "Platform_MacOS"],
            "buildOptions": ["optimize", "inline", "releaseMode", "debugInfo"],
            "lflags": [
                "-rpath", "@executable_path/../Frameworks"
            ]
        },
        "release-win": {
            "buildOptions": ["optimize", "inline", "releaseMode", "debugInfo"],
            "versions": ["Release", "Platform_Windows"],
            "lflags":[
                "-subsystem:windows",
                "-entry:wmainCRTStartup"
            ]
        }
    }
}
```

buildTypes options are merged inline with the rest of your dub.json!

```
$ dub run --build=<...>
```

dub.json

```
    "buildTypes": {
        "development-mac": {
            "versions": ["Development", "Platform_MacOS"],
            "lflags": [
                "-rpath", "@executable_path/../lib-snapshots/dylib"
            ]
        },
        "development-win": {
            "versions": ["Development", "Platform_Windows"]
        },
        "release-mac": {
            "versions": ["Release", "Platform_MacOS"],
            "buildOptions": ["optimize", "inline", "releaseMode", "debugInfo"],
            "lflags": [
                "-rpath", "@executable_path/../Frameworks"
            ]
        },
        "release-win": {
            "buildOptions": ["optimize", "inline", "releaseMode", "debugInfo"],
            "versions": ["Release", "Platform_Windows"],
            "lflags":[
                "-subsystem:windows",
                "-entry:wmainCRTStartup"
            ]
        }
    }
}
```

And, if we specify versions per build type...

Compile-time version check

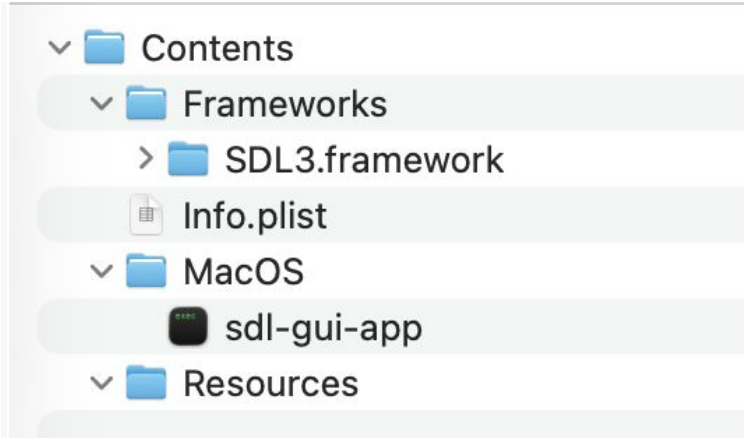
```
version(Development) {
    ASSETS_DIRECTORY = "./assets/";
}
version(Release) {
    string execPath = Runtime.args[0];
    version(Platform_MacOS) {
        ASSETS_DIRECTORY = buildPath(dirName(execPath), "..", "Resources", "assets");
    }
    version(Platform_Windows) {
        ASSETS_DIRECTORY = buildPath(dirName(execPath), "assets");
    }
}
```

Building Universal Binaries

Anatomy of a Mac .app bundle

My Game.app ← this is a directory!

- Contents
  - Resources
    - (your assets)
  - Frameworks
    - SDL3.framework
  - MacOS
    - (your executable)

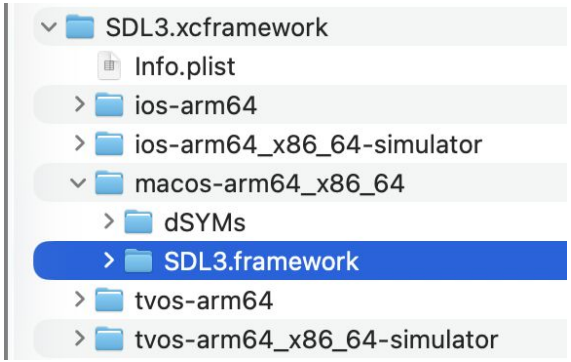


Dynamic libraries?

- They go in /Contents/Frameworks
- Need to be `.dylib` or `.framework`
    - A .framework is also secretly a folder!

Dynamic libraries?

- So why does SDL3 distribute `.xcframework`?
- Answer: an XCode Framework is just a bundle of many `.framework` dirs!
- Just grab the MacOS one :)



Architecture-agnostic executable?

- Remember: we need to support Intel- *and* ARM-based Macs
- Solution: build both, and glue together with `lipo`!
- (This is also done for each dynamic library we are using)

```
dlang-release-mac-x64: $(SOURCES)
# We target 10.13 for compat with older systems
    DFLAGS="-mtriple=x86_64-apple-macosx10.13" dub build
    --build=release-mac --compiler=ldc2 --arch=x86_64
    mv sdl-gui-app sdl-gui-app-x64

dlang-release-mac-arm64: $(SOURCES)
    DFLAGS="-mtriple=arm64-apple-macosx10.13" dub build
    --build=release-mac --compiler=ldc2 --arch=aarch64
    mv sdl-gui-app sdl-gui-app-arm64

mac-appbundle: dlang-release-mac-x64 dlang-release-mac-arm64 Info.plist
# Package x64 and arm64 binaries into a universal binary
    lipo -create -output sdl-gui-app-universal sdl-gui-app-x64
    sdl-gui-app-arm64
```

The “Manifest”

- Every mac .app bundle needs a “manifest” called `Info.plist`

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>CFBundleExecutable</key>
    <string>sdl-gui-app</string>
    <key>CFBundleIdentifier</key>
    <string>com.yoonicode.sdl-gui-app</string>
    <key>CFBundleInfoDictionaryVersion</key>
    <string>6.0</string>
    <key>CFBundleName</key>
    <string>SDL GUI App</string>
    <key>CFBundlePackageType</key>
    <string>APPL</string>
    <key>CFBundleShortVersionString</key>
    <string>0.0.1</string>
    <key>CFBundleVersion</key>
    <string>1</string>
</dict>
</plist>
```

Putting it together (MacOS)

1. Build ARM64 executable
2. Build X64 executable
3. Use lipo to glue ARM64 and X64 binaries into “universal binary”
4. Copy SDL3 and other libs into `Frameworks` folder
5. (Copy any assets into `Resources` folder)
6. Copy Info.plist
7. Copy universal binary into `MacOS` folder
8. Clean up any stray files like `.DS_Store`

Putting it together (MacOS)

```
mac-appbundle: dlang-release-mac-x64 dlang-release-mac-arm64 Info.plist
# Package x64 and arm64 binaries into a universal binary
    lipo -create -output sdl-gui-app-universal sdl-gui-app-x64 sdl-gui-app-arm64

# Make the .app bundle structure
    rm -rf build
    mkdir -p "build/SDL GUI App.app/Contents/MacOS"
    mkdir -p "build/SDL GUI App.app/Contents/Frameworks"
    mkdir -p "build/SDL GUI App.app/Contents/Resources"

# Copy the executable
    cp sdl-gui-app-universal "build/SDL GUI App.app/Contents/MacOS/sdl-gui-app"

# Copy SDL frameworks
# -R is used instead of -r to preserve symlinks withour dereferencing them
    cp -R $(MAC_SDL3_FRAMEWORK_PATH) "build/SDL GUI App.app/Contents/Frameworks"

# Copy INFO plist
    cp Info.plist "build/SDL GUI App.app/Contents"

# Delete .DS_Store directories (they make codesign error out)
    find . -name ".DS_Store" -delete

    open build/
```

Windows

- …copy everything into `lib/` and ship it with your exe.
- That’s it :)

```
win-exe: dlang-release-win
    if exist build rmdir /s /q build
    mkdir build\SDL GUI App-win64
    mkdir build\SDL GUI App-win64\lib
    copy sdl-gui-app.exe build\SDL GUI App-win64\SDL GUI App.exe
    copy "$(WIN_SDL3_DLL_PATH)" build\SDL GUI App-win64\lib\SDL3.dll
    cmd /c start "" build\SDL GUI App-win64
```

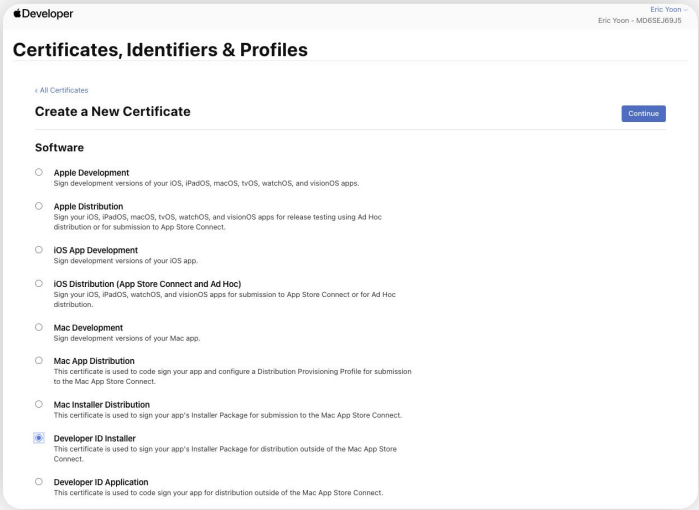
Code Signing & Integrity Checks

Security popups

- Apple will display a security popup if your app is not **notarized and signed**
  - Optimistic view: to protect Mac users from malware
  - Cynical view: to charge you USD $100/year

Step 1: Code Signing

- Need to cryptographically sign our binary
- But first, we need certificates from Apple 🤑



Step 1: Code Signing

- *Entitlements*: a list of requested permissions for your app
- If using LDC2 compiler, you need to allow JIT

sdl-gui-app > sdl-gui-app.entitlements > ...

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
3  <plist version="1.0">
4      <dict>
5          <key>com.apple.security.cs.allow-jit</key>
6          <true/>
7      </dict>
8  </plist>
9
```

Step 1: Code Signing

- From here, you can use XCode CLI tools
- *Remember to sign any frameworks/dylibs you are using too!*

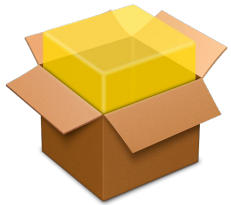
```
codesign --force --verify --verbose --sign "$(DEVELOPER_ID_APPLICATION_CERT_NAME)" --timestamp --options
runtime "build/SDL GUI App.app/Contents/Frameworks/SDL3.framework"
codesign --force --verify --verbose --sign "$(DEVELOPER_ID_APPLICATION_CERT_NAME)" --timestamp --options
runtime --entitlements "sdl-gui-app.entitlements" "build/SDL GUI App.app/Contents/MacOS/sdl-gui-app"
codesign --force --verify --verbose --strict --timestamp --options runtime --entitlements "sdl-gui-app.
entitlements" --sign "$(DEVELOPER_ID_APPLICATION_CERT_NAME)" "build/SDL GUI App.app"
```

Step 2: Notarization

- Code signing solves the *attestation* problem (guarantee binary comes from the app developer)
- However, any developer could ship malicious code!
- Apple wants to scan our executable before we distribute it
- So, we need to upload a copy to Apple servers

Step 2: Notarization

- Apple needs our app in a specific format: a `.pkg` installer



```
productbuild --component "build/SDL GUI App.app" /
Applications --sign "$(DEVELOPER_ID_INSTALLER_CERT_NAME)
" "build/SDL GUI App.pkg"
```

Step 2: Notarization

- Upload the pkg and wait for feedback!
- Staple the notarization to your .app bundle

```
xcrun notarytool submit --apple-id "$(APPLE_ID)" --password "$(APP_SPECIFIC_PWD)"
--team-id "$(APPLE_TEAM_ID)" "build/SDL GUI App.pkg" --wait
echo "Notarization accepted. Waiting 5 seconds for ticket propagation..."
sleep 5
xcrun stapler staple "build/SDL GUI App.pkg"
spctl -a -v --type install "build/SDL GUI App.pkg"
```

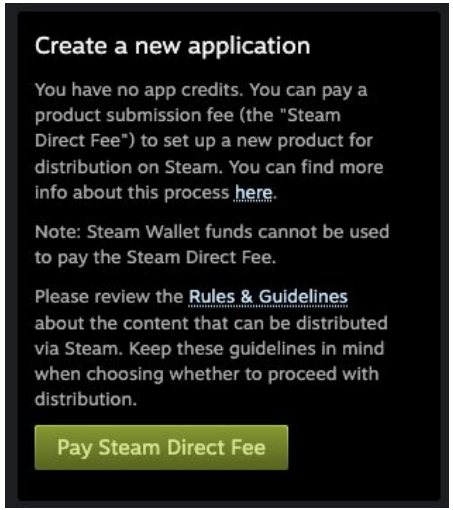
Windows

- Windows does have a code integrity checking framework…
- …but child processes spawned from trusted parent processes are automatically trusted
- Steam is trusted, so we are good

Uploading to Steam

Uploading to Steam

- Only thing left is to upload our builds to Steam!
- You'll need to pay the $100 developer fee to create a Steam app



Steampipe SDK

- **Steampipe** is a CLI build tool that automates uploads
- Configuration for each *depot* lives in a .vdf file

```
"AppBuild"
{
    "AppID" "4285770"
    "Desc" "RGS macos deploy script"

    "ContentRoot" "../RhythmGameStudio/build" // root content folder, relative to location of this file
    "BuildOutput" "build-output" // build output folder for build logs and build cache files

    "Depots"
    {
        "4285772" // MacOS
        {
            "FileMapping"
            {
                "LocalPath" "Rhythm Game Studio.app/*"
                "DepotPath" "Rhythm Game Studio.app" // ensures it stays in a .app folder inside the depot
                "recursive" "1" // include all subfolders
            }
        }
    }
}
```

Steampipe SDK

- Upload your files with steamcmd!
- Smart enough to only upload the diff

```
echo -n "Enter your Steam password for \`yoonicode\`: (input masked)"
read -s STEAM_PASSWORD
echo

echo "Starting Steam upload..."
steamcmd +login "yoonicode" "$STEAM_PASSWORD" +run_app_build "$VDF_PATH" +quit
```

Other storefronts?

- Mac app store: should have no problems!
- Windows app store: only accepts Universal Windows Platform apps (`.appxbundle`)
  - We would need to support Windows ARM64 as well
  - To be figured out later! (Not worth it?)

In Conclusion

- You can now go from `dub run` to deploying on the Steam store page!
- In-depth code examples + written documentation available on GH

github.com/ericyoondotcom/dconf-2026-talk

and now an example of a game using

DLang + SDL3 + Discord SDK + Steamworks SDK...

erh

A no-code game editor designed for creating your own micro beat games

rhythmgamestudio.com

Thank you!

→ Code + docs from this talk: github.com/ericyoondotcom/dconf-2026-talk

→ Check out my work: yoonicode.com

→ Wishlist *Rhythm Game Studio*: rhythmgamestudio.com